UpCharika: Multilingual Clinical AI Assistant

Comprehensive Step-by-Step Project Study Guide & Complete Viva Voce Defense

Project: UpCharika Clinical Healthcare AI	Target Languages: English, Marathi (मराठी), Hindi (हिन्दी)
Architecture: True RAG (Retrieval-Augmented Generation)	Vector Index: 119 MB FAISS Index (Textbook PDFs)
Inference Engine: Groq LPUs (qwen/qwen3.8-27b)	Voice Synthesis: Latency-Optimized gTTS (2.5s)

Part 1: What Makes UpCharika Unique? (Compared to Other Chatbots)

Most college chatbots or generic AI tools (like ChatGPT) are not designed for real-world healthcare in India. Here is a direct comparison showing why UpCharika stands out:

Feature	Generic Chatbots (ChatGPT / Standard College Projects)	UpCharika (Your Project)	Why It Matters
1. Source of Medical Knowledge	Hallucinates or guesses from generic internet data. Can invent dangerous dosages.	Grounded in Verified Medical Literature (RAG): Searches your 119 MB FAISS Textbook Index first before answering.	Prevents medical misinformation; answers cite verified clinical books.
2. Regional Indian Languages	Clunky machine translation that often distorts medical terms in Marathi or Hindi.	Native Devanagari Medical Triage: Pre-built native clinical dictionaries in Marathi (मराठी) and Hindi (हिन्दी) + multilingual LLM synthesis.	In Maharashtra, over 70% of rural/semi-urban patients only understand health advice in Marathi or Hindi.
3. Accessibility for Elderly / Ill Patients	Only text on a screen. Hard for ill or illiterate patients to read long paragraphs.	Instant Spoken Voice Consultation (gTTS): Speaks the critical clinical advice in English, Hindi, or Marathi in under 3 seconds.	Elderly patients or those with poor vision can simply listen to the doctor's advice.
4. Safety & Structure	Wall of unstructured paragraphs. Hard to find what medicine to take or when to rush to the hospital.	Strict 5-Tier Clinical Protocol: Every response is split into: Cause (कारणे), Solution / Diet (उपाय / आहार), Medicine (औषध), When to see Doctor (डॉक्टरांना कधी भेटावे), and Emergency Red Flags (तातडीची मदत).	Patients can immediately spot life-threatening red flags vs home remedies.
5. Offline Resilience & Examiner Proof	Dies completely if internet drops or API quota ends.	Dual-Engine Architecture: If Groq is offline or disabled, it falls back to Direct Textbook Search + Offline Clinical KB. Includes a Test Mode Switch to prove this to examiners.	Reliability during hospital power cuts or network blackouts.

Part 2: Project Structure — How the Files Work Together

Here is the entire anatomy of your project, explained simply:

```
Medical_chatbot/
■
■■■ create_memory_for_llm.py   <-- [Phase 1: The Builder] Converts Medical Books to Math Vectors
■■■ vectorstore/
■   ■■■ db_faiss/             <-- [The Knowledge Vault] 119 MB Searchable Medical Textbook
■       ■■■ index.faiss
■       ■■■ index.pkl
■
■■■ medibot.py                 <-- [Phase 2: The Main Application] Web Server, AI, RAG & Voice
■■■ style.css                  <-- [The Presentation] Clean, modern hospital-grade theme
■■■ users.json                 <-- [Security] User database with encrypted passwords
■■■ requirements.txt           <-- [Dependencies] List of all Python packages needed
■■■ .env / secrets.toml        <-- [Credentials] Secure API keys (Groq)
■■■ venv/                      <-- [Python Virtual Environment] Isolated dependencies
```

File 1: create_memory_for_llm.py (The Builder)

What it does: You run this script once before launching the chatbot. It converts raw medical textbook PDFs into a high-speed search database.
How it works inside (3 steps):
1. Reads PDFs (fitz / PyMuPDF): Opens medical books page-by-page and extracts raw text.
2. Text Chunking (RecursiveCharacterTextSplitter): A whole textbook is too big to search at once. It splits the book into small pieces of 1,000 characters each (with a 100-character overlap so sentences aren't cut mid-thought).
3. Embeddings & FAISS Indexing (sentence-transformers/all-MiniLM-L6-v2): It converts each 1,000-character chunk into a 384-dimensional mathematical vector (a list of 384 numbers representing its clinical meaning) and saves them into vectorstore/db_faiss/index.faiss.

File 2: vectorstore/db_faiss/ (The Medical Knowledge Vault)

• index.faiss (119 MB): The raw mathematical vector matrix created by Facebook AI Research (FAISS). It allows finding the most relevant medical book passage in less than 20 milliseconds.
• index.pkl: The dictionary that maps vector IDs back to the actual English sentences and page numbers in your book.

File 3: medibot.py (The Core Engine)

This is the master file that runs your entire web platform. It handles:
1. User Authentication: Login and Sign Up using bcrypt (passwords are salted and hashed so they can never be stolen).
2. Session Cookies (EncryptedCookieManager): Keeps you logged in even if you refresh your browser.
3. RAG Pipeline (retrieve_medical_context): Embeds the user query and searches db_faiss for top 2 relevant paragraphs.
4. Groq AI Integration (call_groq_chat): Sends the user's question along with textbook paragraphs to Groq LPUs (qwen/qwen3.8-27b).
5. Multilingual Translation (translate_text): Translates responses into Marathi or Hindi dynamically.
6. Optimized Voice Synthesis (generate_voice): Extracts first 2 actionable sentences (extract_voice_summary) and generates audio in 2.5s.
7. Offline / Test Mode Fallback: Bypasses Groq when toggled and delivers direct textbook excerpts with the purple badge 100% Direct Textbook (Groq OFF).

File 4: style.css (The Clinical Design System)

Instead of looking like a generic default Streamlit hobby project, style.css transforms it into a professional Medical SaaS Portal with clean cards, responsive layout, blue/purple/green source badges, rounded input boxes, and smooth buttons.

File 5: users.json (Security)

Holds user records in secure salted hash format: {"nikita": "$2b$12$yv.7Xoamu..."}. Even if someone opens users.json, they cannot see the password because bcrypt uses one-way mathematical encryption with 12 salt rounds.

Part 3: Deep Walkthrough of a Single Question

Let's trace exactly what happens inside the computer when a user types: 'What is jaundice and what diet should I eat?'

```
Step 1: Patient hits Enter in medibot.py
   ■
   ▼
Step 2: Vector Search (RAG Step)
   • The sentence is converted into 384 numbers by `all-MiniLM-L6-v2`.
   • FAISS searches through 119 MB of vectors in 15 milliseconds.
   • It finds Textbook Page 42:
     'Exposure to people with jaundice; contaminated foods; alcohol; duration;
      anorexia, abdominal pain... evaluation of nutritional status...'
   ■
   ▼
Step 3: Augmenting the AI Prompt
   • medibot.py constructs a prompt:
     'You are UpCharika. Here is verified medical textbook context:
      [Textbook Paragraphs on Jaundice and Nutrition].
      Formulate a clinical triage answer for the user's query.'
   ■
   ▼
Step 4: Groq AI Inference
   • Groq LPU processes this in 1.2 seconds.
   • Generates the structured 5-part clinical response.
   ■
   ▼
Step 5: Voice Summary & Audio Generation
   • `extract_voice_summary` extracts:
     'Jaundice occurs due to elevated bilirubin levels and liver inflammation.
      Adhere to a light, easily digestible diet with plenty of fluids and boiled vegetables.'
   • `gTTS` converts these 2 concise sentences to MP3 in 2.2 seconds.
   ■
   ▼
Step 6: Display on Screen
   • Card rendered with badge: ■ Medical Textbook Grounded + Groq AI
   • 5 color-coded sections: Cause, Solution, Medicine, Doctor, Emergency.
   • Audio player ready for instant playback.
```

Lesson 1: How the Medical Memory was Created (create_memory_for_llm.py)

This file is an offline pipeline. You run it once to convert medical PDF textbooks into the 119 MB FAISS vector database. It has 3 core functions:

```
[Raw PDFs in data/]
        ■
        ▼ (Function 1: load_pdf_files)
[Extracted Text Pages + Metadata]
        ■
        ▼ (Function 2: create_chunks)
[1,000-character Text Chunks with 100 overlap]
        ■
```

```
        ▼ (Function 3: store_in_faiss)
[384-dimensional Vectors stored in vectorstore/db_faiss]
```

Step 1: Loading the PDFs (load_pdf_files, Lines 17–51)

```
import fitz  # PyMuPDF library

for pdf_path in pdf_files:
    with fitz.open(pdf_path) as doc:
        for page_num, page in enumerate(doc):
            text = page.get_text('text').strip()
            if text:
                metadata = {
                    'source': os.path.basename(pdf_path),
                    'page': page_num + 1
                }
                documents.append({'text': text, 'metadata': metadata})
```

• What fitz (PyMuPDF) does: PyMuPDF is one of the fastest PDF extractors in Python. It opens the medical textbook, iterates through every single page, and extracts clean text.
• Why metadata is saved: Every page records which book it came from and the exact page number. When someone asks a question, the system knows not just the answer, but the textbook page where it was found!

Step 2: Chunking the Text (create_chunks, Lines 53–69)

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100
)
chunks = text_splitter.create_documents(texts, metadatas=metadatas)
```

Why do we need chunking? (Crucial Viva Question)
Examiner Question: 'Why not just embed the entire textbook or whole chapters at once?'
Your Answer: 'Because embedding models have a maximum token limit (typically 256–512 tokens). If you put an entire 50-page chapter in, the model forgets specific details. By splitting into 1,000-character chunks, each chunk represents a single focused medical concept (like symptoms of Jaundice or fever management).'

What is chunk_overlap=100?
Imagine a critical sentence spans across the boundary: Chunk 1 ends at: 'For severe dehydration, oral rehydration' ... and Chunk 2 starts at: 'salts (ORS) should be given immediately.' If there were zero overlap, that medical advice would be cut in half and lost! The 100-character overlap ensures that the end of Chunk 1 and the start of Chunk 2 share the same words, preserving full sentence meaning.

Step 3: Vector Embeddings & FAISS (store_in_faiss, Lines 71–85)

```
embedding_model = HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')

db = FAISS.from_documents(
    documents=text_chunks,
    embedding=embedding_model
)
db.save_local('vectorstore/db_faiss')
```

What are Embeddings? (The Core Math)
• An embedding model is a neural network that translates human text into numbers.
• all-MiniLM-L6-v2 converts every chunk into an array of 384 decimal numbers (a 384-dimensional vector).
• Words with similar clinical meanings have high mathematical similarity (calculated via Cosine Similarity or Euclidean Distance):

Similarity('high fever', 'elevated body temperature') → 0.92
Similarity('high fever', 'stock market crash') → 0.05

What is FAISS?
• FAISS stands for Facebook AI Similarity Search.
• Instead of comparing a question to all 100,000 sentences one by one (which would take 10 seconds), FAISS uses an indexed tree structure to find the closest vectors in under 20 milliseconds.

Lesson 2: How the Live Chatbot Runs (medibot.py)

Now let's examine the 4 most important functions inside medibot.py:

1. Retrieving Textbook Context (retrieve_medical_context, Lines 60–75)

```
@st.cache_resource(show_spinner=False)
def get_vectorstore():
    embed = HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')
    return FAISS.load_local('vectorstore/db_faiss', embed, allow_dangerous_deserialization=True)

def retrieve_medical_context(query, k=2):
    db = get_vectorstore()
    docs = db.similarity_search(query, k=k)
    return '\n\n'.join([d.page_content.strip() for d in docs])
```

• @st.cache_resource: Ensures the 119 MB vector database is loaded into RAM only once. Every user query reuses the database in memory without reloading.
• k=2: Retrieves the Top 2 most relevant paragraphs matching the patient's question.

2. The AI Generation & Prompt Injection (get_ai_response, Lines 455–550)

```
context = retrieve_medical_context(prompt, k=2)

prompt_to_groq = f'''
You are UpCharika, a clinical AI healthcare assistant.
Provide structured, professional medical guidance using:
Cause:
Solution:
Medicine:
When to see doctor:
Emergency:

Verified Medical Reference Context (from Clinical Textbook):
\"\"\"
{context}
\"\"\"
Carefully incorporate and ground your clinical advice in the above textbook findings.
'''
```

• This is True RAG: We don't ask the AI model to guess. We feed it the exact textbook paragraphs inside {context} and tell it: 'Ground your answer in this textbook.'
• Groq processes this prompt and generates the final response in ~1.2 seconds.

3. Low-Latency Voice Optimization (extract_voice_summary, Lines 355–375)

```
def extract_voice_summary(text):
    sentences = re.split(r'(?<=[.!?■])\s+', text.strip())
    summary = ' '.join(sentences[:2])
    return summary[:250]
```

Why was this needed?

• In earlier versions, gTTS tried to convert the entire 1,500-character document into voice. It took 82 seconds and froze the screen!

• By isolating the first 2 actionable sentences (under 250 characters), synthesis time dropped from 82 seconds to 2.5 seconds!

4. Direct Textbook Mode (Groq Offline / Test Mode)

```
if force_offline:
    # Bypasses Groq completely!
    # Returns the direct excerpt from the FAISS vector database
    return direct_textbook_body, 'textbook_direct'
```

• When you check [] Test Mode: Disable Groq, the system bypasses the LLM and delivers the textbook excerpt directly with the purple badge 100% Direct Textbook (Groq OFF).

SECTION 1: The 6 Core Concepts Explained Deeply

1. What is RAG (Retrieval-Augmented Generation)?

• The Problem: If you ask a standard AI (like ChatGPT) about a disease, it answers from memory. If it forgets or doesn't know, it hallucinates (invents believable but fake medical advice).

• The Solution (The 'Open-Book Exam' Analogy):

- Standard AI without RAG = A student writing a closed-book exam from memory. They might misremember a dosage or symptom.

- UpCharika with RAG = An open-book exam. When the patient asks a question, UpCharika first opens the medical textbook, finds the exact 2 paragraphs, and then writes the answer based only on those textbook pages.

2. What are Vector Embeddings?

• In Simple Words: Computers do not understand words; they only understand numbers.

• The Concept: An embedding model (sentence-transformers/all-MiniLM-L6-v2) converts a sentence into 384 decimal numbers. These 384 numbers represent the conceptual meaning of the sentence.

• Why this is powerful (Semantic Search vs Keyword Search):

- Old search (Google in 1998): If you search 'high body temperature', it only looks for pages with the exact words 'high', 'body', 'temperature'. If the book says 'fever', it misses it!

- Embedding search (UpCharika): The model knows that 'high body temperature' and 'acute fever' have nearly identical numbers in 384-dimensional space. It finds the right page even if different words are used!

3. What is Chunking & Chunk Overlap?

• Chunking (chunk_size=1000): A 500-page medical textbook has over 1,000,000 words. You cannot feed the whole book into an embedding model at once. So, we split the book into bite-sized pieces of 1,000 characters each (about 150 words — roughly one medical paragraph).
• Chunk Overlap (chunk_overlap=100): What if a crucial sentence is split right down the middle across two chunks? Example: Chunk 1 ends with: 'For severe dehydration, oral rehydration' ... and Chunk 2 starts with: 'salts (ORS) should be given.' Without overlap, the meaning is cut in half. The 100-character overlap ensures that the end of Chunk 1 repeats at the start of Chunk 2, so no medical advice is ever severed.

4. What is FAISS and How is it Different from SQL?

• FAISS stands for Facebook AI Similarity Search.
• Traditional Database (MySQL / PostgreSQL): Searches for exact matches: WHERE disease = 'jaundice'. Cannot understand similarity or context.
• Vector Database (FAISS): Searches for geometric proximity between vectors. Given a question vector Q, FAISS calculates the mathematical distance (Cosine Similarity) between Q and all textbook vectors, returning the closest matches in under 20 milliseconds.

5. Why Groq and What is an LPU?

• Traditional AI runs on GPUs (Graphics Processing Units made by Nvidia). GPUs are powerful, but they take 5 to 15 seconds to generate an answer.
• Groq invented the LPU (Language Processing Unit): A chip designed specifically for sequential language generation. It generates text at 250–400 tokens per second (sub-second response time). That is why UpCharika responds in 1.5 seconds!

6. Why bcrypt Password Hashing?

• If a hacker steals your users.json, storing plain passwords (e.g. password123) is a critical security flaw.
• Even standard SHA-256 is vulnerable to 'rainbow table' brute-force attacks.
• bcrypt adds a random 'Salt' (random garbage characters) to every password before hashing and applies 12 rounds of mathematical stretching. It is mathematically impossible to reverse a bcrypt hash back into the password.

SECTION 2: Top Most Asked Questions & Answers (Viva Prep)

Q1: 'Why did you use RAG instead of Fine-Tuning the model on medical data?'

Your Answer:
"Fine-tuning changes the internal weights of a model, which requires expensive GPU training, takes days, and can still hallucinate. Moreover, if medical guidelines change, you must re-train the whole model. With RAG, the LLM stays untouched. We simply update the PDF in our vector database without any re-training. It is faster, free, zero-GPU, and 100% verifiable because we can see the exact textbook source."

Q2: 'What is the difference between an Embedding Model and an LLM?'

Your Answer:
"They have two completely different jobs: 1. The Embedding Model (all-MiniLM-L6-v2) is a Search Engine. It converts text into 384 numbers so we can find relevant textbook paragraphs. It cannot write or speak. 2. The LLM (Groq / Qwen / LLaMA) is a Writer / Brain. It reads the retrieved textbook paragraphs and writes a fluent, conversational clinical answer in English, Marathi, or Hindi."

Q3: 'What mathematical metric does FAISS use to find similar vectors?'

Your Answer:
"FAISS measures similarity using either Cosine Similarity (the angle between two vectors) or L2 Euclidean Distance (the straight-line distance between two points in 384-dimensional space). The smaller the distance or the closer the cosine angle is to 1, the more medically relevant that textbook passage is."

Q4: 'What is temperature=0.3 in your Groq API call?'

Your Answer:
"Temperature controls the creativity vs. determinism of the AI: • A high temperature (0.8 to 1.0) makes the AI creative and unpredictable (good for poetry or story writing). • A low temperature (0.1 to 0.3) makes the AI strict, factual, and cautious. In medical healthcare, we set temperature=0.3 so the model stays strictly grounded in verified facts and does not take creative risks."

Q5: 'How did you solve the voice generation delay (gTTS lag)?'

Your Answer:
"Initially, Google Text-to-Speech was trying to synthesize the entire 1,500-character clinical diagnosis into audio, which caused an 82-second freeze. I created an intelligent extraction algorithm (extract_voice_summary) that extracts only the first 2 primary clinical sentences (under 250 characters). This reduced voice synthesis time from 82 seconds down to 2.5 seconds."

Q6: 'How do you prove that an answer came from your textbook and not from Groq's general training?'

Your Answer:
"We implemented a Test Mode Switch in the sidebar (Test Mode: Disable Groq). When enabled, it turns off Groq completely. The system executes the FAISS vector search alone and displays the raw medical textbook excerpt with a purple badge: 100% Direct Textbook (Groq OFF). When Test Mode is turned off, Groq refines that exact text with the blue badge: Medical Textbook Grounded + Groq AI."

Q7: 'What happens if the internet goes down or Groq API is unavailable?'

Your Answer:
"UpCharika has a 3-tier fault tolerance system: 1. If Groq is active -> True RAG (Textbook + LLM). 2. If Groq fails but textbook has the topic -> Direct Textbook Retrieval. 3. If offline and asking about common symptoms (fever, headache, diabetes, acidity) -> Instant Offline Knowledge Base in English, Marathi, and Hindi."

Q8: 'How does the Multilingual Translation work?'

Your Answer:
"UpCharika uses a hybrid approach: • For common clinical conditions, we have pre-translated native Devanagari clinical triage cards in Marathi and Hindi (zero latency). • For open-ended questions, Groq generates the response directly in native Marathi/Hindi using specialized Devanagari system prompts. • As a fallback, deep-translator (Google Translator engine) re-translates dynamically if the user toggles the sidebar language."